

 **MANUAL TÉCNICO V1.0**



Linux AI Ops Studio

Plataforma autónoma de administración y monitoreo de servidores Linux
potenciada por inteligencia artificial

VERSIÓN

1.0.0

AUTOR

Daniel Godoy

FECHA

Mayo 2026

LICENCIA

Apache 2.0

Contenido

01	Introducción y Visión General	— stack tecnológico, arquitectura	3
02	Vault de Credenciales SSH	— gestión segura de servidores	5
03	Dashboard de Monitoreo	— telemetría en tiempo real	7
04	Intelligence Center	— terminal, logs, SFTP, seguridad	10
05	Multiagente Copilot IA	— agentes autónomos, modos de control	14
06	API REST	— endpoints, contratos, ejemplos	17
07	Configuración y Despliegue	— variables de entorno, producción	19
08	Auditoría de Seguridad (CIS Hardening)	— 22 controles, compliance	21

STACK TECNOLÓGICO

React 19

TypeScript 5

Tailwind CSS 4

Express 5

Vite 6

SQLite (sql.js)

ssh2

OpenRouter AI

Framer Motion



Introducción y Visión General

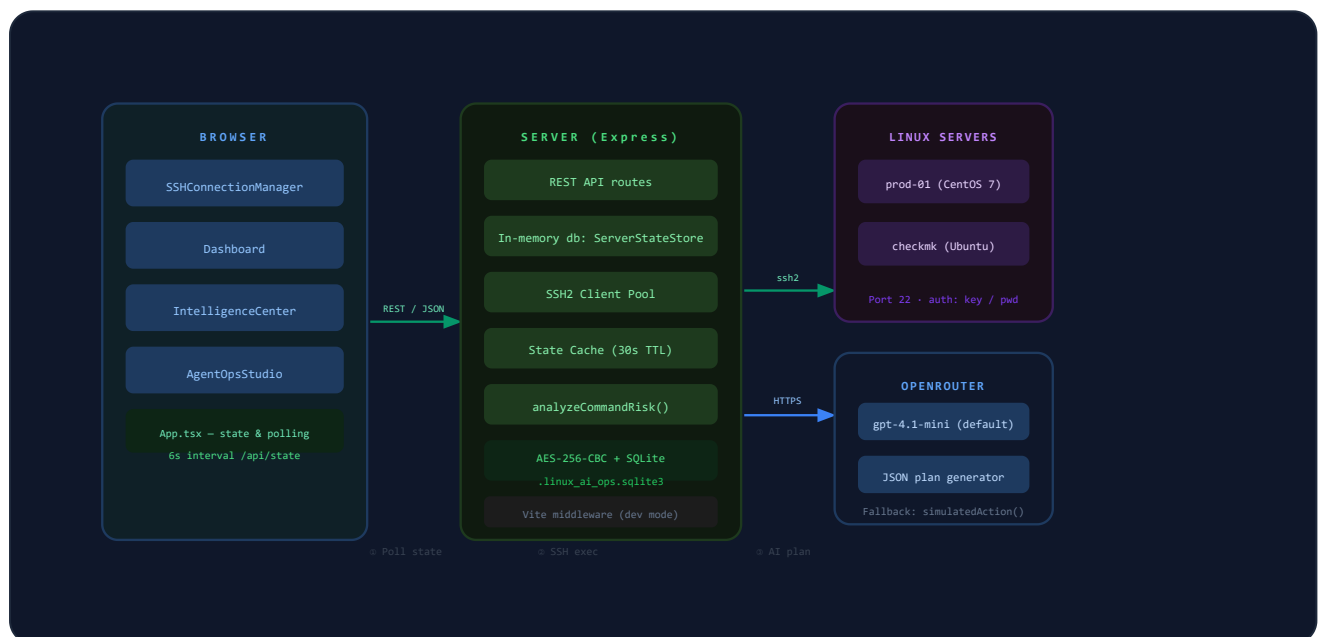
Linux AI Ops Studio es una plataforma full-stack de administración autónoma de servidores Linux. Combina telemetría en tiempo real, gestión de procesos y servicios, terminal SSH, explorador SFTP, auditoría de seguridad CIS y un sistema multiagente de IA — todo en una interfaz web unificada.



Propósito central

Reducir el tiempo de respuesta ante incidentes Linux desde minutos a segundos. El agente IA genera planes de acción auditables con control granular de riesgo (bajo / medio / alto) antes de ejecutar cualquier comando.

Arquitectura del Sistema



Principios de Diseño



Monolito intencional

Todo el backend vive en `server.ts`. Sin microservicios. Despliegue de 1 proceso.



Polling + Cache

El frontend sondea estado cada 6s. El servidor cachea las respuestas SSH 30s para no saturar las conexiones.



Credenciales cifradas

Las contraseñas y claves privadas se almacenan en SQLite con cifrado AES-256-CBC antes de escribirlas a disco.



IA con control de riesgo



Dual mode SSH



Responsive

Cada comando generado por IA recibe un nivel low/medium/high antes de ejecutarse. El usuario aprueba o rechaza.

Modo simulado para demos (IPs mock), modo real para producción. Sin diferencia de interfaz para el usuario.

3 columnas en desktop (xl:), vista de panel único con tabs en mobile. Sin frameworks de UI externos.



El **SSH Connection Vault** es el punto de entrada de la plataforma. Administra un pool de servidores Linux con sus credenciales de acceso, persistidas cifradas en SQLite y con un asistente de 6 pasos para incorporar nuevos nodos de forma segura.



Fig. 2.1 — Vista general de la aplicación. El Vault ocupa la columna izquierda (1/5 del ancho).

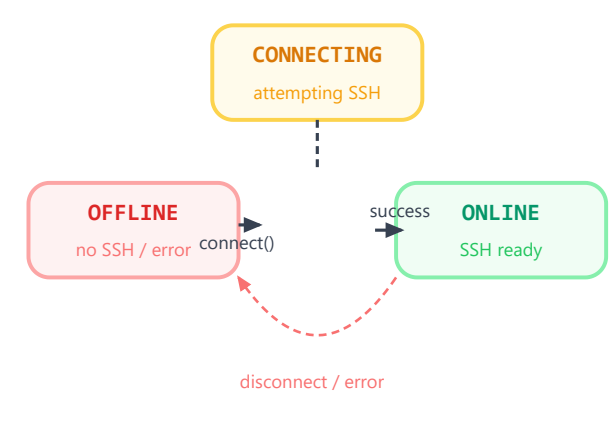
Asistente de Conexión (6 pasos)

- 1 Identidad del servidor**
Nombre descriptivo, dirección IP/hostname y puerto SSH (por defecto 22).
- 2 Credenciales de usuario**
Nombre de usuario Linux. Soporte para usuarios no-root con escalación posterior vía `sudo` o `su`.
- 3 Método de autenticación**
Selección entre **contraseña** o **clave privada PEM**. La clave se analiza automáticamente (RSA, ECDSA, ED25519, DSA, OpenSSH, PKCS8).
- 4 Verificación de huella**
El cliente calcula y muestra el fingerprint SHA-256 y MD5 de la clave privada para auditoría.
- 5 Test de conectividad**
Llamada `POST /api/servers/test-connection` que verifica acceso SSH real antes de guardar.
- 6 Guardar en Vault**
Las credenciales se cifran AES-256-CBC y persisten en `.linux_ai_ops.sqlite3`. El servidor queda disponible para todas las sesiones futuras.

Gestión de servidores

Acción	Descripción
Conectar	Establece sesión SSH y comienza polling de estado cada 6s
Desconectar	Marca servidor como offline, limpia caché de estado
Seleccionar	Activa el servidor en Dashboard e Intelligence Center
Eliminar	Borra del Vault y de la base de datos SQLite
Limpiar unused	Elimina masivamente servidores sin actividad reciente

ESTADOS DE CONEXIÓN



Seguridad de credenciales

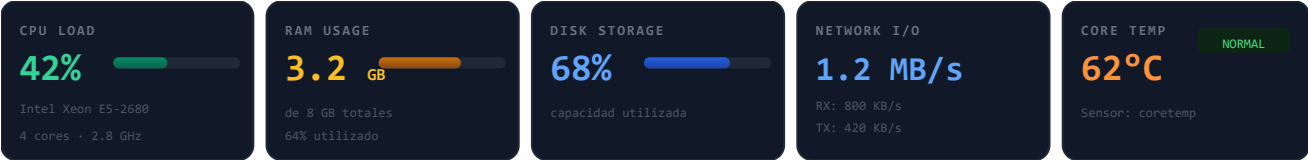
Las contraseñas y claves privadas nunca viajan en texto plano al cliente. En el Vault solo se almacena el nombre, host, puerto y tipo de auth. Las credenciales salen del servidor únicamente para establecer la conexión SSH.



Dashboard de Monitoreo

El Dashboard centraliza toda la telemetría del servidor activo: métricas de hardware en tiempo real, procesos, servicios systemd y contenedores Docker. Los datos se sincronizan cada 6 segundos vía `GET /api/servers/:id/state`.

Panel de métricas principales



Gráficos de actividad en tiempo real

Live CPU Activity

Historial de hasta 20 muestras visualizado como wave bar chart. Colores adaptativos: verde (<60%), ámbar (60-85%), rojo (>85%). Los datos se mantienen solo en memoria del navegador — se resetean al recargar.

Memory Resource Commit

Barra de progreso *commit charge* con valores absolutos en GB. Muestra **used** vs **free available** en tiempo real. Alimentado por `/proc/meminfo` vía SSH.

Tabs del Dashboard

Tab	Contenido	Acciones disponibles
Monitor de Recursos	CPU wave chart + RAM commit bar + métricas en tiempo real	Solo lectura — auto-refresh
Procesos	Tabla completa de procesos con PID, USER, CPU%, MEM%, TTY, COMMAND	SIGKILL por PID individual
Servicios	Listado de unidades systemd/SysV/OpenRC con estado active/inactive	Restart Stop por servicio
Docker Containers	Contenedores con imagen, estado, puertos expuestos y uso de recursos	Start Stop por container

EJEMPLO — VISTA DE PROCESOS

PID	USER	CPU%	MEM%	TTY	COMMAND	OP.
1	root	0.0%	0.2%	?	/usr/lib/systemd/systemd	SIGKILL
1243	www-data	2.1%	4.8%	?	nginx: worker process	SIGKILL
2891	mysql	8.3%	18.2%	?	/usr/sbin/mysqld	SIGKILL



CAPÍTULO 04

Intelligence Center

El Intelligence Center agrupa en 4 tabs la interacción avanzada con el servidor: terminal SSH interactiva, visor de logs del sistema, explorador y editor SFTP, y auditoría de seguridad CIS. Cada tab se activa sin recargar estado — comparten la misma sesión SSH.



Bash Console

Terminal SSH con historial de comandos, escalación root/sudo, output capeado a 500 KB.



SysLog

Logs del sistema con filtros por nivel (all/error/warning/info) y diagnóstico IA.



SFTP Browser

Explorador de archivos con upload/download, editor online y explicación IA del código.



Security

Auditoría CIS con 22 controles, clasificación PASS/FAIL/WARN, comandos de remediación.

4.1 Terminal Bash (SSH)

La consola es una réplica funcional de una terminal bash remota. Soporta:

Función	Descripción
Historial ↕	Navegación por comandos previos de la sesión
CWD tracking	Rastreo del directorio actual tras cada <code>cd</code>
Output cap	500 KB máx. por comando; truncado con aviso
2000 líneas	Buffer circular — las más antiguas se descartan
Escalación su	Modal de contraseña → sesión root persistente
Escalación sudo	Modal por comando — no persiste credenciales
Color coding	stdout: blanco · stderr: rojo · system: cyan

```
Linux AI Ops Smart Console [v1.0.0]

[prod-01:~]$ df -h
Filesystem Size Used Avail Use%
/dev/sda1 80G 54G 26G 68% /

[prod-01:~]$ systemctl status nginx
● nginx.service - nginx HTTP Server
Active: active (running)

[prod-01:~]$
```

4.2 SFTP Browser



Fig. 4.1 — SFTP Browser. Explorador de directorios con upload drag-and-drop y editor inline.

El SFTP Browser navega el sistema de archivos del servidor remoto mediante la API `/api/servers/:id/sftp/*`.
Características principales:

Navegación de directorios

Listado con tipo (archivo/directorio), tamaño, permisos y fecha de modificación. Breadcrumb clickeable.

Upload con cola

Hasta 3 uploads concurrentes. Barra de progreso individual, velocidad en bytes/s, cancelable con XHR abort.

Download inteligente

Archivos <500 MB: descarga buffereada. Archivos ≥500 MB: redirección al endpoint nativo del browser.

Editor inline

Abre archivos de texto directamente en el browser. Edita y guarda con `PUT /api/servers/:id/sftp/put`.

Explicación IA

Botón "Explicar con IA" envía el contenido del archivo a OpenRouter para obtener descripción en español.

Límites de seguridad

El editor carga máx. 100 KB. Archivos binarios o de gran tamaño se descargan directamente.

4.3 Security — Auditoría CIS Hardening

Fig. 4.2 — Security tab. Vista vacía antes de ejecutar el escaneo de hardening.

El módulo de seguridad ejecuta un script bash de una sola pasada SSH que recopila 23 parámetros del sistema y los evalúa contra controles CIS Benchmark y NIST 800-53.

ID	Control	Severidad	Compliance
SSH-001	Root login SSH deshabilitado	HIGH	CIS 5.2.8 · NIST AC-6
SSH-002	Autenticación por contraseña deshabilitada	HIGH	CIS 5.2.13 · NIST IA-5
SSH-003	Timeout sesión inactiva ≤300s	MEDIUM	CIS 5.2.12
SSH-004	MaxAuthTries ≤4	MEDIUM	CIS 5.2.7
SSH-005	PermitEmptyPasswords = no	HIGH	CIS 5.2.9
SSH-006	X11 Forwarding deshabilitado	LOW	CIS 5.2.6
FW-001	Firewall activo (UFW / firewalld)	HIGH	CIS 3.5
FW-002	Reglas DROP/REJECT en iptables INPUT	MEDIUM	NIST SC-7
OS-001	ASLR habilitado (=2)	HIGH	CIS 1.5.3
OS-002	SELinux / AppArmor activo	HIGH	CIS 1.6
OS-003	IP forwarding deshabilitado	MEDIUM	CIS 3.1.1
OS-004	Suid core dumps deshabilitados	MEDIUM	CIS 1.5.1
ACC-001	Sin usuarios UID=0 adicionales a root	HIGH	CIS 5.4.2
ACC-002	Sin entradas NOPASSWD en sudoers	HIGH	NIST AC-6
ACC-003	Sin contraseñas vacías o débiles	HIGH	CIS 6.2.1

ID	Control	Severidad	Compliance
FS-001	Sin archivos world-writable en /etc, /bin	MEDIUM	CIS 6.1.10
FS-002	Archivos SUID innecesarios	MEDIUM	CIS 6.1.13
SVC-001	Sin servicios inseguros (telnet, rsh, tftp)	HIGH	CIS 2.1



Multiagente Copilot IA

El **Multiagente Copilot** es el componente estrella de la plataforma. Integra OpenRouter para convertir instrucciones en lenguaje natural en planes de ejecución auditables. Cada plan se presenta paso a paso con nivel de riesgo y requiere aprobación explícita del operador antes de ejecutar comandos sensibles.

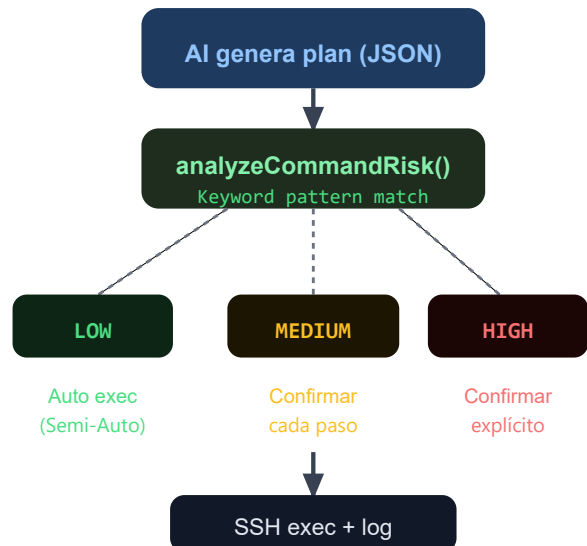
Agentes disponibles



Modos de control de autonomía

Modo	Comportamiento
Sugerencia	Solo genera el plan. Nunca ejecuta comandos sin que el usuario cambie de modo. Ideal para explorar opciones.
Semi-Autónomo	Ejecuta automáticamente pasos de riesgo low . Los de riesgo medium y high requieren aprobación individual.
Totalmente Autónomo	Ejecuta todos los pasos del plan sin interrupciones. Requiere confianza máxima en el agente.

FLUJO DE APROBACIÓN DE COMANDOS



Contrato JSON de respuesta IA

El endpoint `POST /api/openrouter/run-agent` espera que el modelo devuelva exactamente este formato. El parser `tryParseJson()` tolera respuestas envueltas en markdown (````json ... ````):

```
// Estructura del plan generado por OpenRouter
{
  "explanation": "Diagnóstico completo de disco y filesystem del servidor.",
  "riskLevel": "low", // low | medium | high
  "steps": [
    {
```

```
"id": "step-1",
"title": "Uso de disco por partición",
"description": "Lista el espacio libre y usado en todas las particiones.",
"command": "df -h",
"risk": "low"
},
{
  "id": "step-2",
  "title": "Top 10 directorios más grandes",
  "command": "du -sh /* 2>/dev/null | sort -rh | head -10",
  "risk": "low"
}
],
"reportSummary": "El sistema tiene 68% de disco usado en /dev/sda1. Se recomienda..."
}
```



Fallback sin API key

Si `OPENROUTER_API_KEY` no está configurada, el servidor usa `generateSimulatedAgentAction()` que devuelve un plan cableado según el rol activo. La interfaz funciona completamente en modo demostración.



Todos los endpoints sirven y consumen JSON. El servidor corre en `http://localhost:3005` por defecto. No hay autenticación — diseñado para uso en red local o detrás de un proxy inverso con auth.

Servidores y conexiones

GET	/api/servers	Lista todos los servidores del vault
POST	/api/servers	Registra un nuevo servidor (name, host, port, username, authType, password?, privateKey?)
DELETE	/api/servers/:id	Elimina servidor del vault y de SQLite
POST	/api/servers/:id/connect	Marca servidor como conectando e inicia SSH
POST	/api/servers/:id/disconnect	Desconecta y limpia caché de estado
POST	/api/servers/test-connection	Verifica credenciales SSH sin guardar
POST	/api/servers/cleanup-unused	Elimina masivamente servidores sin actividad

Estado y telemetría

GET	/api/servers/:id/state	Estado completo: server, processes, services, docker, logs, cronJobs, securityAssets (cacheado 30s)
POST	/api/servers/:id/execute	Ejecuta comando SSH → { stdout, stderr, exitCode, riskLevel }
GET	/api/servers/:id/sftp/ls	Lista directorio SFTP (?path=...) → { entries[] }
GET	/api/servers/:id/sftp/get	Descarga archivo (?path=...) → stream binario
POST	/api/servers/:id/sftp/put	Sube o actualiza archivo (multipart/form-data)

Seguridad y AI

POST	/api/servers/:id/hardening-scan	Ejecuta script CIS bash → { policies[], summary }
POST	/api/openrouter/run-agent	Genera plan de acción IA → { explanation, riskLevel, steps[], reportSummary }
GET	/api/openrouter/status	Verifica conexión a OpenRouter → { connected, model }

Ejemplo de respuesta — /api/servers/:id/state

```
// GET /api/servers/srv-abc123/state
{
  "server": {
    "id": "srv-abc123",
    "name": "prod-01",
    "host": "192.168.1.100",
    "status": "online",
    "cpuUsage": 42,
    "ramUsage": 3.2, // GB
    "ramTotal": 8, // GB
    "diskUsage": 68, // %
    "temperature": 62, // °C
    "uptime": "up 48 weeks, 1 day",
    "osName": "CentOS Linux 7 (Core)",
    "rxRate": 800, "txRate": 420 // KB/s
  },
  "processes": [/* LinuxProcess[] */],
  "services": [/* SystemService[] */],
  "dockerContainers": [/* DockerContainer[] */],
  "logs": [/* LogLine[] */],
  "cronJobs": [/* CronJob[] */],
  "securityAssets": [/* SecurityAsset[] */]
}
```



Variables de entorno

Variable	Requerida	Por defecto	Descripción
OPENROUTER_API_KEY	No	—	Activa IA real. Sin ella usa respuestas simuladas.
OPENROUTER_MODEL	No	gpt-4.1-mini	Modelo de IA. Puede ser cualquier modelo de OpenRouter.
OPENROUTER_API_BASE	No	https://openrouter.ai/api/v1	URL base de OpenRouter (override para proxies).
STORAGE_DB_PATH	No	.linux_ai_ops.sqlite3	Ruta del archivo SQLite con credenciales cifradas.
PORT	No	3005	Puerto HTTP del servidor Express.

Instalación y arranque

```
# 1. Clonar y preparar
git clone <repo> linux-ai-ops-studio
cd linux-ai-ops-studio

# 2. Instalar dependencias
npm install

# 3. Configurar entorno (opcional)
cp .env.example .env
# Editar .env y añadir OPENROUTER_API_KEY

# 4. Desarrollo (Vite + Express en un proceso)
npm run dev
# → http://localhost:3005

# 5. Producción
npm run build # Genera dist/ (frontend) + dist/server.cjs (backend)
npm start    # Sirve dist/ como estáticos
```

Estructura de archivos del proyecto

```
linux-ai-ops-studio/
├─ server.ts      # Backend Express completo (monolítico)
├─ src/
```

```

├─ App.tsx # Root: state, polling, callbacks
├─ types.ts # Tipos compartidos (no hay otras compartidas)
├─ components/
│   ├─ SSHConnectionManager.tsx # Vault + wizard 6 pasos
│   ├─ Dashboard.tsx # Telemetría + procesos/servicios/docker
│   ├─ IntelligenceCenter.tsx # Terminal + Logs + SFTP + security
│   ├─ AgentOpsStudio.tsx # Multiagente IA copilot
│   └─ MetricsGraphics.tsx # CPUUsageMeter, RAMUsageMeter, CircularProgress
├─ .linux_ai_ops.sqlite3 # DB en tiempo de ejecución (gitignored)
├─ .env # Variables de entorno (gitignored)
├─ vite.config.ts
├─ tailwind.config.ts
└─ tsconfig.json

```

Verificación de tipos

```
npm run lint # tsc --noEmit - sin test suite, esto es la única validación estática
```



Nota de producción

El estado en memoria (`db`) se resetea al reiniciar el servidor, excepto las conexiones que se recargan desde SQLite. Para persistencia completa de logs, procesos, etc., se requeriría una base de datos externa.

Modos de ejecución SSH



Modo simulado

Se activa automáticamente cuando el host es `192.168.1.10` o `192.168.1.12` (IPs mock) o cuando no hay credenciales. `runSimulatedCommand()` devuelve respuestas cableadas para los comandos más comunes de Linux.



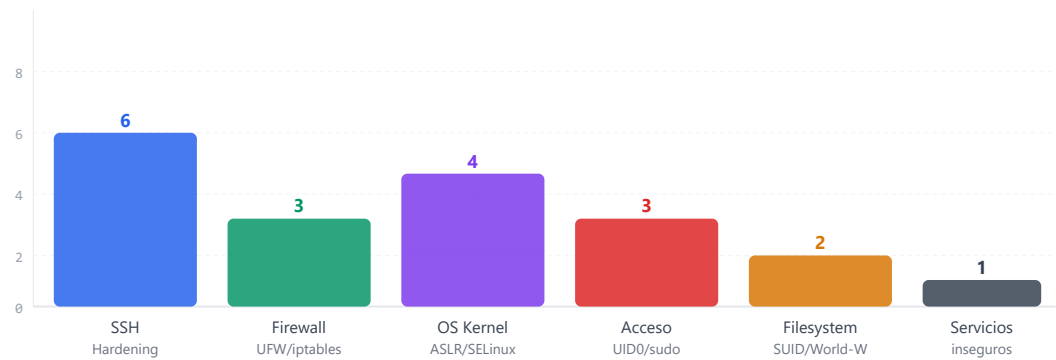
Modo SSH real

Para cualquier otro host con credenciales válidas. Usa la librería `ssh2` para ejecutar comandos reales. Compatible con CentOS 7, Ubuntu 20/22, Debian 11/12, RHEL 8/9.



El módulo de hardening ejecuta 22 comprobaciones en una sola llamada SSH. El script bash recoge los valores en crudo y el servidor los normaliza a objetos `HardeningPolicy` con PASS/FAIL/WARN, risk_score (0-100) y comandos de remediación listos para copiar y ejecutar.

CATEGORÍAS DE CONTROL — DISTRIBUCIÓN DE CHECKS



Niveles de severidad

Cada control tiene un **risk_score** de 0 a 100. Los controles con FAIL suman al puntaje total de riesgo del servidor.

Severidad	risk_score FAIL	Categoría
HIGH	70–95 puntos	SSH, Firewall, ASLR, Cuentas
MEDIUM	40–65 puntos	Timeouts, Permisos, Kernel params
LOW	10–30 puntos	X11, Configuraciones menores

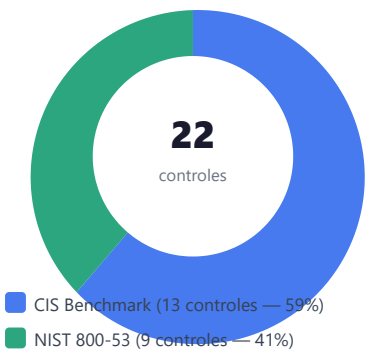
Remediación asistida

Para cada control con FAIL, el sistema proporciona el comando exacto de corrección:

```
# SSH-001: Deshabilitar root Login
sed -i 's/^PermitRootLogin.*/PermitRootLogin no/' \
/etc/ssh/sshd_config
systemctl restart sshd

# SSH-002: Deshabilitar auth por contraseña
sed -i 's/^PasswordAuthentication.*/PasswordAuthentication no/' \
```

FRAMEWORKS DE COMPLIANCE CUBIERTOS



Análisis IA post-scan

Tras ejecutar el scan, el botón "Analizar con IA" envía todos los resultados a OpenRouter para obtener un resumen ejecutivo con prioridades de remediación en español.

```
/etc/ssh/sshd_config
systemctl restart sshd
```

Resumen de capacidades

Módulo	Tecnología clave	Estado
SSH Vault con cifrado	AES-256-CBC + sql.js (SQLite)	✓ Producción
Telemetría en tiempo real	Polling 6s · /proc/stat · free -g · df	✓ Producción
Gestión de procesos	ps aux · kill -9	✓ Producción
Servicios systemd/SysV	systemctl · service init.d · rc-service	✓ Producción
Docker containers	docker ps · docker start/stop	✓ Producción
Terminal SSH interactiva	ssh2 exec · root/sudo escalation	✓ Producción
SFTP Browser + Editor	sftp subsystem · XHR upload · stream download	✓ Producción
CIS Hardening Audit	Bash one-shot script · 22 controles	✓ Producción
Multiagente IA	OpenRouter · gpt-4.1-mini · JSON planner	✓ Producción
Test suite automatizado	—	No implementado
Autenticación de usuario	—	No implementado

Linux AI Ops Studio · Versión 1.0.0 · Apache License 2.0
Desarrollado por **Daniel Godoy** · Mayo 2026
npm run dev → http://localhost:3005